

Report 3 - CUDA

September 29, 2026

0.1 Introduction

The plan is to convert the picture to grayscale by replacing the R,G,B values of each pixel with their average. I will try with different picture sizes to compare the gap.

- Load an image from file (matplotlib's imread)
- Flatten image into 1D array of RGB (reshape(pixelCount, 3))
- Implement grayscale using CPU (for range)
- Implement grayscale using GPU
- Save/show the image after each grayscale to validate the result visually, dtype=np.uint8
- Use time.time() to measure speedup

0.2 Work

0.2.1 Importing data

Importing data, checking the size and showing it with pyplot to ensure the import worked correctly.

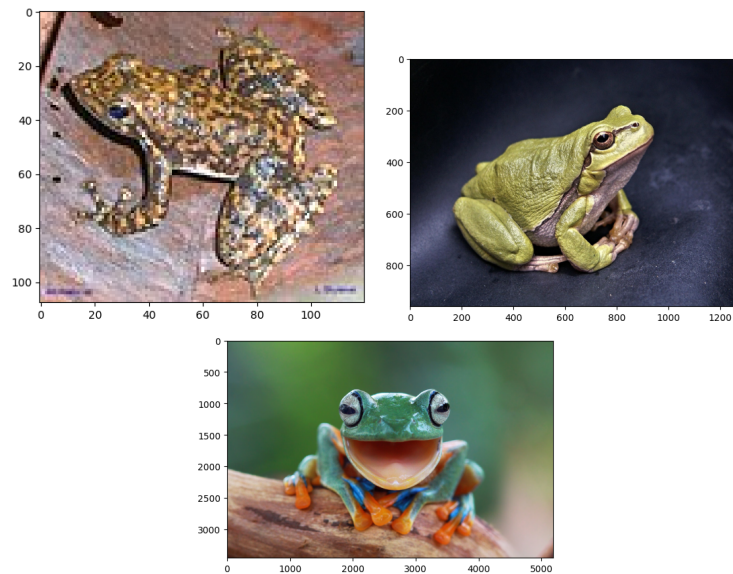
```
from numba import cuda
from matplotlib import pyplot

img = pyplot.imread("image.jpg")

height = len(img)
width = len(img[0])
pixelCount = height*width
print(height, "*", width, "=", pixelCount, " pixels")

pyplot.imshow(img)
pyplot.show()
```

1. Output: $108 * 120 = 12960$ pixels (13k)
2. Output: $960 * 1280 = 1228800$ pixels (1.2M)
3. Output: $3456 * 5184 = 17915904$ pixels (18M)



$$g(y, x) = \frac{1}{3} \sum_{i=0}^3 I(y, x, i)$$

0.2.2 Grayscale with CPU

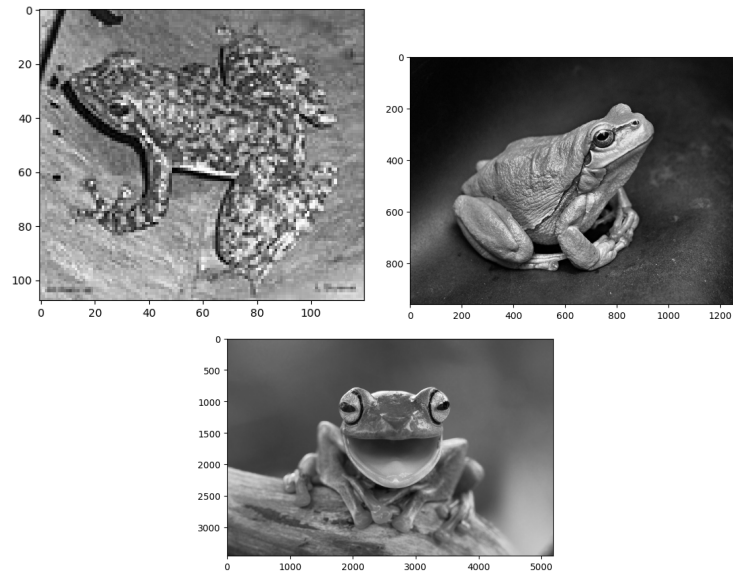
Importing the remaining missing dependencies. Implementing grayscale following this formula: I was having issues with the types (Python moment) and a uint8 overflow on the intermediary sum (in the average calculation) so I had to make (R,G,B) uint16 before converting the avg back to uint8 for my 8-bit RGB.

```
import numpy as np
import time
```

```
#initializing the result array /w uint8 dtype (default=float)
gray_array = np.zeros((pixelCount, 3), np.uint8)
```

```
def grayscale_CPU(r_img):
    for i in range(pixelCount):
        #making the types real clear as an overflow was causing noise on the final i
        (r,g,b)=np.int16((r_img[i,0], r_img[i,1], r_img[i,2]))
        gray = np.uint8((r+g+b)/3)
        gray_array[i,0] = gray_array[i,1] = gray_array[i,2] = gray
    return gray_array
```

```
r_img = img.reshape(pixelCount, 3)
```



```

start=time.time()
gray_array = grayscale_CPU(r_img)
stop=time.time()
print("duration",stop-start)

gray_img = gray_array.reshape(height , width , 3)

pyplot.imshow(gray_img)
pyplot.show()

```

1. Output: duration 0.04524660110473633
2. Output: duration 9.251688241958618
3. Output: duration 70.6480085849762

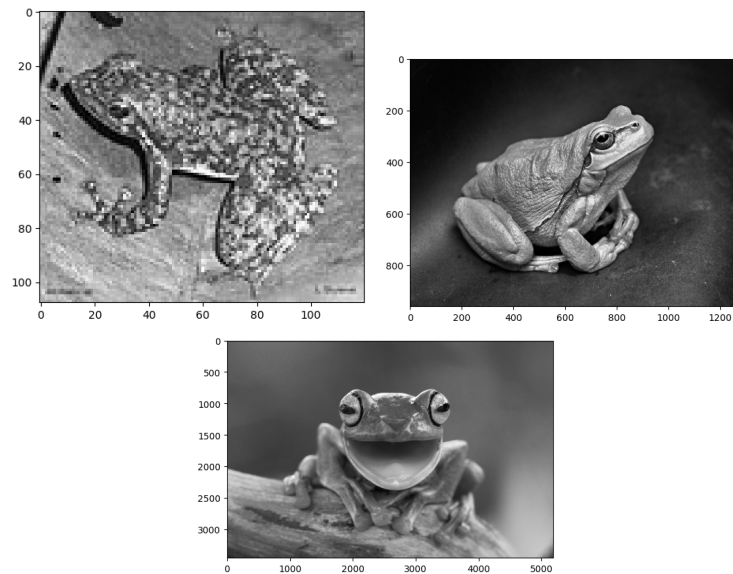
0.2.3 Grayscale with GPU

Based on Prof. implementation, added the round up of gridSize.

```

@cuda.jit
def grayscale(src , dst):
    # where are we in the input?
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    g = np.uint8((src[tidx , 0] + src[tidx , 1] + src[tidx , 2]) / 3)
    dst[tidx , 0] = dst[tidx , 1] = dst[tidx , 2] = g

```



```
devSrc = cuda.to_device(r_img)
devDst = cuda.device_array_like(r_img)
blockSize = 32

#rounding up
gridSize = (pixelCount + blockSize - 1) // blockSize

start=time.time()
#kernel call
grayscale[gridSize, blockSize](devSrc, devDst)
stop=time.time()
print("duration",stop-start)

gray_img = devDst.copy_to_host().reshape(height, width, 3)

pyplot.imshow(gray_img)
pyplot.show()
```

1. Output: duration 0.542658805847168
2. Output: duration 0.3926730155944824
3. Output: duration 0.39334869384765625

0.3 Conclusion

CPU vs GPU computation

1. 0.04 vs 0.54
2. 9.25 vs 0.39
3. 70.64 vs 0.39

The bigger the load (more pixels) the higher the gap between CPU and GPU speed. On a small picture (13k pixels) the CPU was **10x faster** than the GPU. However, on a big picture (18M pixels), the CPU was roughly **200x slower** than the GPU.